

OMNI-GRAPH

AI for Industrial Knowledge Intelligence: Unified Asset & Operations Brain

ET AI Hackathon 2026 — Problem Statement 8

Team Konoha

Role	Name
Team Leader	Nithish Chandra S
Team Member	Hrithik Sankar R

Document: Technical Submission — PRD · TRD · Backend Schema · Application Flow · Implementation Runbook
Version: 1.0.0

Table of Contents

- 1. Product Requirements Document (PRD)
- 2. Technical Requirements Document (TRD)
- 3. Backend Schema
- 4. Application Flow
- 5. Implementation Plan — 48-Hour Sprint Execution Runbook
- 6. Architecture Diagram

1. Product Requirements Document (PRD)

1.1 Executive Summary

Industrial plants run their entire operational memory across two disconnected artifacts: a 500-page PDF maintenance manual, and a P&ID spatial schematic. When a technician needs the torque spec, lockout procedure, or compliance code for valve V-104, that knowledge exists somewhere in the first document, and the valve itself exists somewhere in the second — and nothing connects them except

a human who happens to remember, or the time it takes to search by hand.

Omni-Graph closes that gap. It ingests both documents simultaneously, uses a Hybrid-Edge AI pipeline to extract equipment entities, maintenance rules, and spatial coordinates, and fuses them into a single queryable knowledge graph. Click the valve on the schematic — get the exact rule paragraph and compliance code back in milliseconds, with the 2D drawing, the 3D graph, and the context drawer updating in lockstep. No vector search, no similarity-score guessing, no hallucinated citations — every answer is a deterministic graph traversal back to a real document paragraph and a real pixel coordinate.

1.2 Vision

To build **Omni-Graph**, an air-gapped, Hybrid-Edge AI intelligence platform that eradicates the "knowledge cliff" in asset-intensive industries — the moment a plant's most experienced hands retire and decades of undocumented, tribal operational knowledge leave with them, while the only surviving record is a static PDF nobody has time to search under pressure.

Omni-Graph's architecture is deliberately split along a trust boundary: entity and rule extraction from proprietary maintenance manuals runs **on-device**, on a local, lightweight LLM — the enterprise's engineering documentation is parsed and reasoned over without ever leaving the plant network in plaintext. Only the comparatively low-sensitivity task of locating tags on a schematic image is offloaded to a cloud vision engine. This "reason locally, see in the cloud" split is what makes a genuinely **Hybrid-Edge** — not just "cloud with a demo toggle" — architecture possible on commodity hardware.

Roadmap beyond the hackathon build:

- Multi-site federation — one Neo4j instance per plant, federated queries across a fleet.
- Predictive maintenance signals layered onto the graph (failure-rate correlation by equipment class and rule-compliance history).
- Voice and AR front-ends for hands-free technician queries on the plant floor.
- Full on-prem NVIDIA NIM container deployment (self-hosted, not the hosted API) for customers whose security posture requires zero outbound calls of any kind.

1.3 Use Cases — The Visakhapatnam Narrative

Consider a scenario representative of daily operational reality across India's dense industrial corridors — clusters like Visakhapatnam, home to integrated steel plants, petrochemical refineries, and major port infrastructure operating around the clock.

It's 2 a.m. A night-shift technician is flagged to an anomaly on pump P-201A. The senior engineer who wrote the maintenance SOP for this exact unit retired eighteen months ago. The only surviving

record is a 400-page manual in a site office drawer, and the P&ID schematic pinned to a wall three buildings away. Somewhere in that manual is the correct lockout/tagout sequence and the torque spec for the flange bolts; somewhere on that schematic is the exact valve the SOP refers to. Neither artifact points to the other.

With Omni-Graph already deployed against this facility's documentation, the technician instead opens a tablet, clicks P-201A on the digital schematic, and in under two seconds has the LOTO sequence, the 45 Nm torque spec, and the governing compliance code (e.g. OSHA 1910.147) on screen — sourced back to the exact paragraph it came from.

This class of knowledge fragmentation — the gap between "the procedure exists" and "the procedure is findable in the moment it's needed" — is a well-documented contributing factor behind delayed emergency response and preventable safety incidents across asset-intensive industry generally, not a hypothetical risk. Omni-Graph is designed to close exactly this gap.

Secondary use case — Institutional knowledge capture: before a senior engineer retires, their annotated manuals and as-built schematics are run through Omni-Graph once, converting decades of undocumented plant-specific judgment into a permanent, queryable graph asset rather than an exit interview nobody has time to conduct properly.

1.4 Competitive Moat — Air-Gapped Enterprise Security

- **Deterministic graph traversal, not vector RAG.** Every answer is a Cypher traversal to a real node with a real source paragraph and real coordinates — not a nearest-neighbor embedding match that can confidently return the wrong paragraph.
- **On-device text reasoning.** The local LLM (Ollama, Llama-3.2-3B) parses the enterprise's proprietary maintenance manuals entirely on-premises. The manual text itself never crosses the network boundary during extraction — the only outbound call in the entire pipeline is the vision-localization request, and that payload is pixel geometry, not manual content.
- **Demo-safety circuit breaker doubles as a real field capability.** Mock Mode was built to survive a hackathon venue's Wi-Fi dying mid-pitch — but the same mechanism is exactly what a plant on a segmented or intermittently-connected OT network needs: if the link to the cloud vision engine drops, the system degrades to a local replay path rather than failing outright.
- **No single point of failure.** Four independently restartable services (frontend, Go router, Python worker, Neo4j) — Neo4j is written to as a best-effort mirror and is never on the critical read path, so a database hiccup never takes down a technician's query.
- **Near-zero marginal cost.** A quantized 3B model resident on commodity edge hardware, a narrowly-scoped cloud vision call, and open-source Neo4j Community Edition — no per-seat LLM licensing, no vector database subscription.

2. Technical Requirements Document (TRD)

2.1 System Architecture

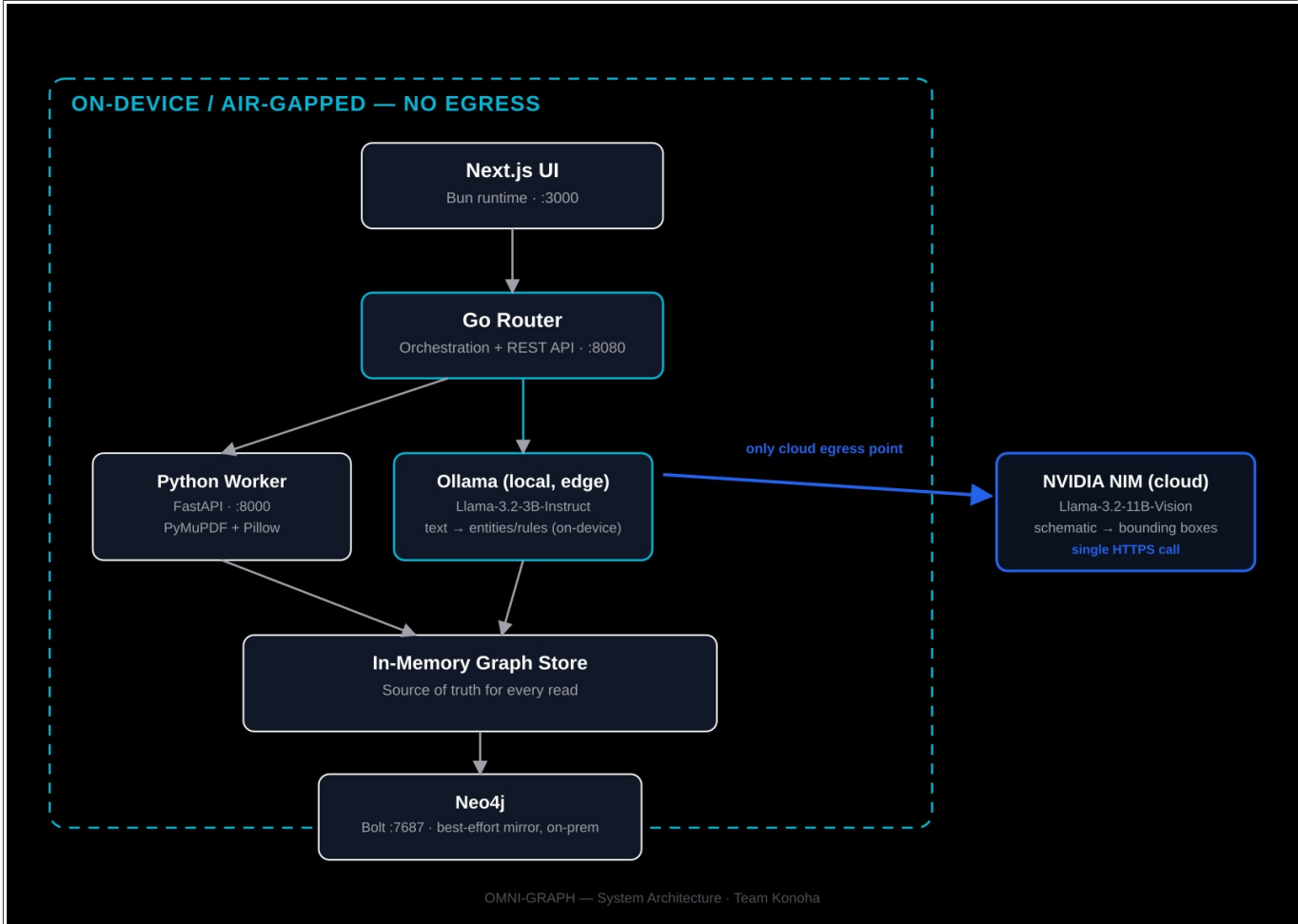


Figure 1 — System architecture. Everything inside the dashed boundary runs on-device with no outbound calls; NVIDIA NIM is the single cloud egress point.

Service	Tech	Port	Role	Trust boundary
frontend	Next.js + Bun	3000	Ingestion wizard, 2D schematic, 3D graph, intelligence drawer	Local network
backend	Go (stdlib net/http)	8080	REST API, orchestration, in-memory store, model clients	Local network
worker	Python (FastAPI)	8000	PDF text extraction/chunking, image downscale	Local network
ollama	Llama-3.2-3B-Instruct	11434	Text entity/rule extraction	On-device — no egress
neo4j	Neo4j	7474 /	Durable graph mirror	Local network

	Community	7687		
NVIDIA NIM	Llama-3.2-11B-Vision	HTTPS	Spatial bounding-box localization	Only cloud egress point

Why split the model layer this way: Meta designed the Llama-3.2 1B/3B tier specifically for on-device and edge deployment, and the 11B-Vision tier for workloads that need materially more compute than a 4GB-VRAM edge box can offer. Omni-Graph's architecture follows that design intent exactly — the small model stays local, the heavy multimodal model is the one component that bursts to a cloud endpoint.

Why Go for the router: goroutines make concurrent fan-out to the local model trivial (one goroutine per text chunk, rate-limited by a buffered-channel semaphore) without the callback/async-await ceremony a Node backend would need for equivalent concurrency.

Why a separate Python worker: PyMuPDF (PDF parsing) and Pillow (image processing) have no mature Go equivalents; isolating them behind an HTTP boundary keeps the Go router free of cgo/FFI and lets the worker restart or scale independently.

Why the in-memory store is the source of truth, not Neo4j: every read the frontend makes needs sub-50ms latency and zero external dependencies. Neo4j is written to as a best-effort mirror on every ingest — if it's down, ingestion and every UI interaction still work, just without a durable Cypher-queriable copy until it recovers.

2.2 Hardware Constraints — The 4GB VRAM Edge Target

Target hardware: 16GB system RAM, 4GB VRAM (e.g. an NVIDIA RTX 3050-class laptop GPU), or a CPU-only edge box as a slower but fully functional fallback for the local model.

Component	Footprint	Location
Ollama + Llama-3.2-3B-Instruct (Q4_K_M quantization)	~2.0–2.2 GB resident	Edge device
Neo4j Community Edition (JVM)	~500 MB – 1 GB	Edge device (Docker)
Go router + Python worker	<300 MB combined at idle	Edge device
NVIDIA NIM — Llama-3.2-11B-Vision-Instruct	0 local footprint (hosted)	NVIDIA cloud

Hard design rule (NFR): heavy vision-model weights must never be scheduled onto local VRAM. Only the quantized 3B instruction model is resident on-device; everything the 11B-Vision model does happens on NVIDIA's infrastructure, reached over a single outbound HTTPS call per ingest.

Latency/rate targets:

- NVIDIA NIM calls capped at <40 requests/minute.
- Local Ollama fan-out uses a 5-slot semaphore — bounded by CPU/VRAM contention on the edge box.
- End-to-end graph read targets <200ms, backed by the in-memory store, not a live Neo4j round-trip.

2.3 AI Pipeline

1. **Upload.** Frontend posts `manual` (PDF) + `schematic` (PNG/JPG/DWG) as multipart form data to `POST /api/v1/ingest`.
2. **Parse (Python worker).** PyMuPDF extracts raw text and splits it into overlapping chunks (4,000 chars, 400-char overlap, paragraph-aligned). Pillow concurrently downscales the schematic to fit within 2048×2048 (JPEG, quality 85).
3. **Local text extraction (Ollama).** The Go router fans the text chunks out concurrently to the on-device Ollama endpoint. Llama-3.2-3B-Instruct returns structured JSON: equipment tags, names, systems, and their maintenance/safety/compliance rules. This step never leaves the local network.
4. **Spatial localization (NVIDIA NIM).** Once entities are back, the Go router sends the compressed schematic and the real extracted tag names to NVIDIA NIM — grounding the vision model against known tags measurably improves localization.
5. **Bounding-box validation.** The pipeline detects degenerate responses (every box has zero area) and retries with a coarser 3×3-region prompt, trading pixel precision for reliability.
6. **Merge & persist.** The Go router deduplicates entities by equipment tag, unions their rules, writes to the in-memory store, and best-effort mirrors into Neo4j via a single UNWIND/MERGE Cypher transaction.
7. **Live log stream.** Every stage ([SYS], [PY], [OLLAMA], [NVIDIA], [GRAPH], [NEO4J]) emits a buffered log line surfaced through `GET /api/v1/logs`.

Mock Mode — the air-gap circuit breaker. `MOCK_MODE=true` makes ingestion skip the worker, Ollama, and NVIDIA NIM entirely and replay a pre-computed Golden Dataset payload with a realistic ~1.5s simulated delay — a live-demo Wi-Fi failure becomes a feature, not a crash.

3. Backend Schema

3.1 Why Neo4j

Industrial knowledge is inherently relational: a single pump connects to a specific rule, and that rule

connects to a specific paragraph, and the pump also connects to a specific spatial coordinate on a drawing. Neo4j lets the backend traverse `(Equipment) - [:HAS_RULE] -> (Rule)` in milliseconds and return exactly the subgraph the UI needs.

3.2 Node Labels

Label	Key Properties	Represents
:Equipment	tag_id (unique), name, system	A physical asset extracted from text or drawing
:Rule	rule_id (unique), category, description, source_paragraph	A maintenance/safety/compliance rule bound to an asset
:Coordinate	coord_id (unique), x_min, y_min, x_max, y_max	A bounding box locating an asset on a schematic

3.3 Relationships (Edges)

```
(:Equipment {tag_id})-[:HAS_RULE]->(:Rule {description, category, rule_id, source_paragraph})
```

```
(:Equipment {tag_id})-[:LOCATED_AT]->(:Coordinate {x_min, y_min, x_max, y_max, coord_id})
```

Both Rule and Coordinate nodes are MERGED on their content, so re-ingesting the same document is idempotent — it never creates duplicate nodes.

3.4 Constraints & Indices

```
CREATE CONSTRAINT unique_equipment IF NOT EXISTS FOR (e:Equipment) REQUIRE e.tag_id IS UNIQUE;
```

```
CREATE CONSTRAINT unique_rule IF NOT EXISTS FOR (r:Rule) REQUIRE r.rule_id IS UNIQUE;
```

```
CREATE CONSTRAINT unique_coord IF NOT EXISTS FOR (c:Coordinate) REQUIRE c.coord_id IS UNIQUE;
```

```
CREATE INDEX equipment_tag_idx IF NOT EXISTS FOR (e:Equipment) ON (e.tag_id);
```

```
CREATE INDEX rule_category_idx IF NOT EXISTS FOR (r:Rule) ON (r.category);
```

3.5 Bulk Insertion (One Transaction Per Ingest)

```
UNWIND $payload.entities AS entity
```

```
MERGE (e:Equipment {tag_id: entity.equipment_tag})
```

```
ON CREATE SET e.name = entity.name, e.system = entity.system
```

```
|
```

```

WITH e, entity
UNWIND entity.rules AS rule
MERGE (r:Rule {description: rule.description})
ON CREATE SET r.category = rule.category,
r.rule_id = randomUUID(),
r.source_paragraph = rule.source_paragraph
MERGE (e)-[:HAS_RULE]->(r)
|
WITH e, entity
UNWIND entity.spatial AS space
MERGE (c:Coordinate {x_min: space.x_min, y_min: space.y_min,
x_max: space.x_max, y_max: space.y_max})
ON CREATE SET c.coord_id = randomUUID()
MERGE (e)-[:LOCATED_AT]->(c)

```

3.6 Context-Drawer Traversal (Used on Every Node Click)

```

MATCH (e:Equipment {tag_id: $tag_id})
OPTIONAL MATCH (e)-[:HAS_RULE]->(r:Rule)
OPTIONAL MATCH (e)-[:LOCATED_AT]->(c:Coordinate)
RETURN e.tag_id AS Tag, e.name AS Name,
collect(DISTINCT r) AS Rules,
collect(DISTINCT c) AS Coordinates

```

4. Application Flow

Omni-Graph's frontend is a four-route Next.js application; each route is one deliberate step in the ingestion-to-insight journey, sharing a single cross-pane selection state. All screenshots below are captured from the actual running build, not mockups.

4.1 / — Landing

Hero section introducing Omni-Graph, with a single call to action — "Enter Command Center" — that routes to /ingest.

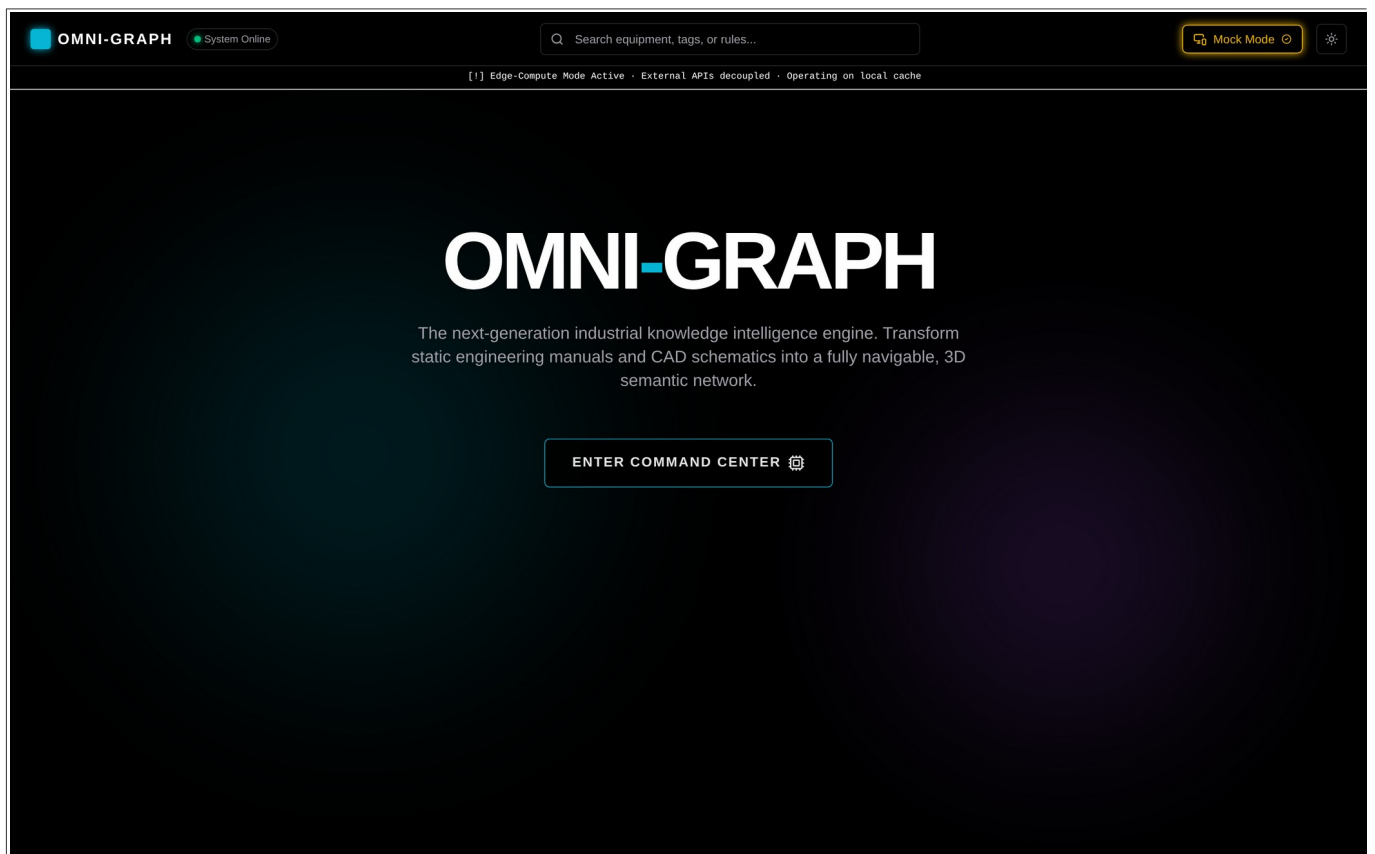


Figure 2 — Landing page.

4.2 /ingest — Data Ingestion Hub

A sequential two-step dropzone wizard: Step 1 accepts the spatial schematic (PNG/JPEG/DWG), Step 2 accepts the technical manual (PDF). Each completed step collapses to a compact confirmation state before the "Initialize AI Extraction" button appears.

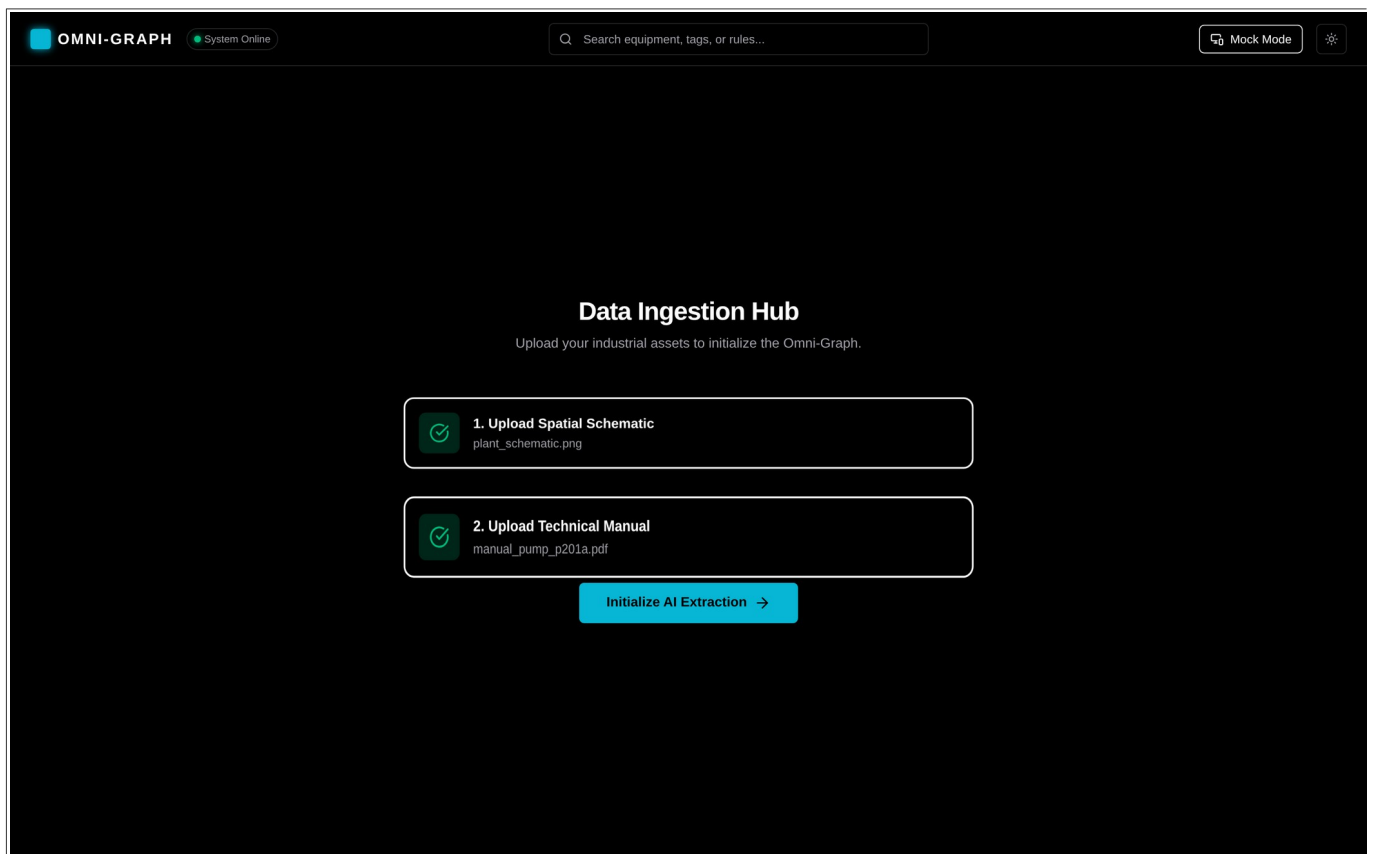


Figure 3 — Ingestion hub with both real files staged (plant_schematic.png + manual_pump_p201a.pdf).

4.3 /process — Processing Terminal

A split view that makes the AI pipeline visible instead of hiding it behind a spinner: a schematic-scan animation on the left, and a live terminal on the right replaying real tagged pipeline log lines as the actual backend call executes.

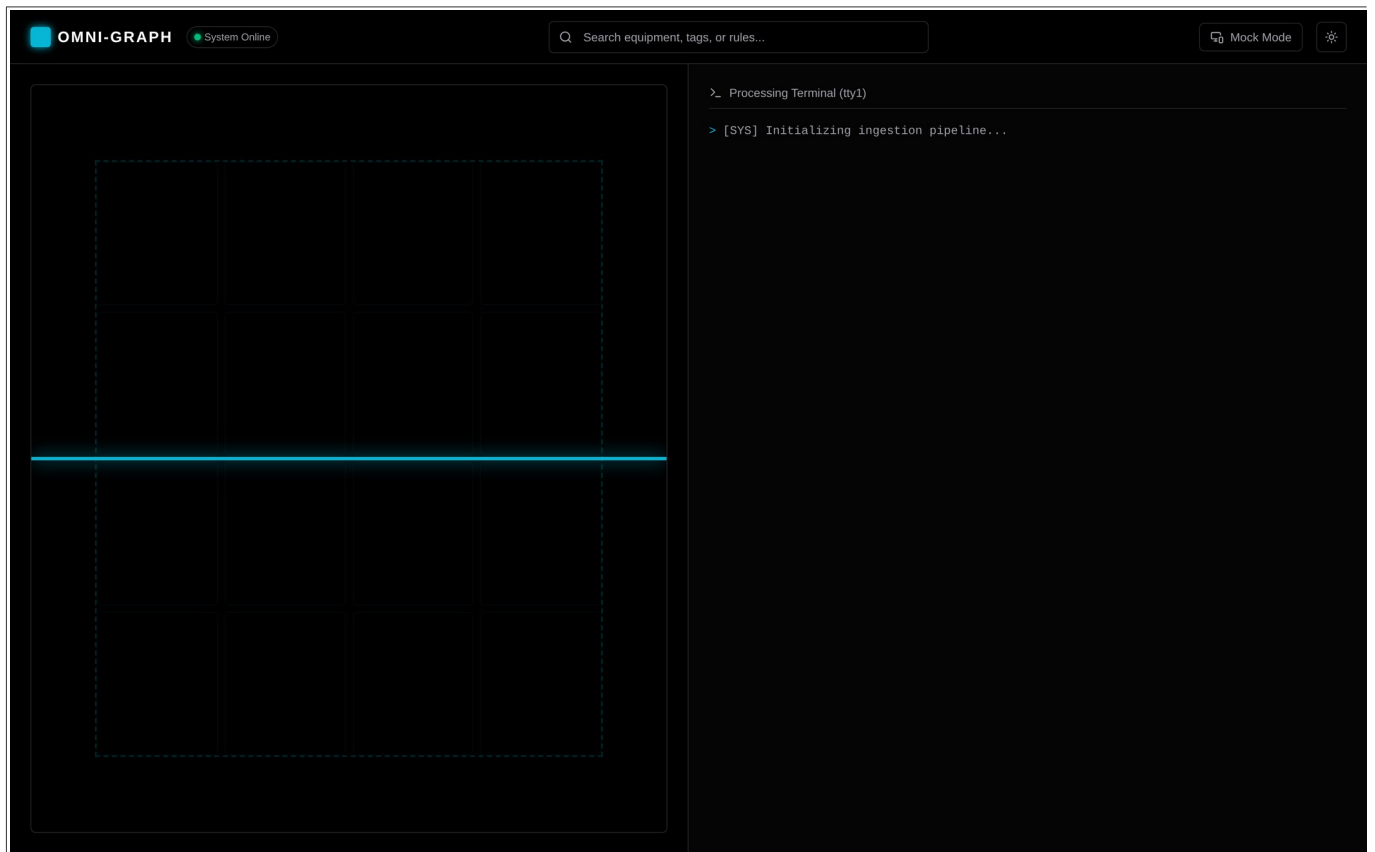


Figure 4 — Live processing terminal mid-pipeline.

4.4 /graph — The Omni-Graph Canvas

A 50/50 split view plus an overlay drawer: **BlueprintCanvas** renders the real uploaded schematic image with real NVIDIA-located bounding boxes; **OmniGraph3D** renders the WebGL force-directed knowledge graph; the **IntelligenceDrawer** slides in with the real rules, audit-trail source paragraphs, and linked assets for whichever node is active.

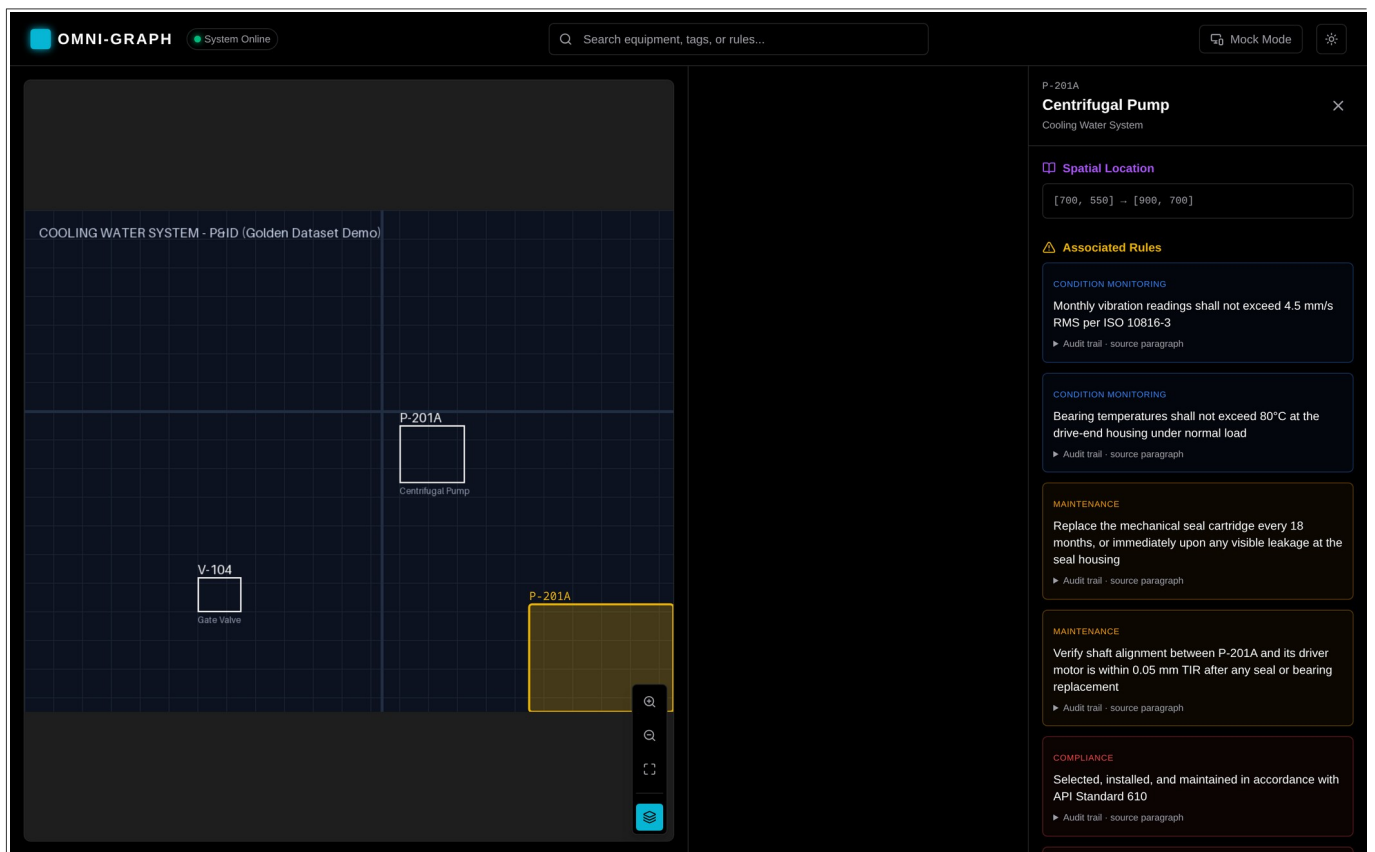


Figure 5 — Live result after a real ingest: P-201A selected, showing real extracted rules (ISO 10816-3 vibration limit, seal replacement interval, shaft alignment tolerance, API 610) sourced directly from the uploaded manual, and a real NVIDIA-located bounding box on the real uploaded schematic.

4.5 Cross-Pane Synchronization

A single Zustand field, `activeNodeId`, is the one source of truth for "what's selected." Click a bounding box on the schematic → the 3D graph camera flies to the matching node and dims unrelated ones → the drawer opens and fetches `GET /api/v1/node/{tag}/context`. Click a node in the 3D graph → the schematic highlights the matching bounding box. Exactly one place holds selection state; every pane is a pure function of it.

4.6 Mock Mode — The Demo-Safety Flow

If external connectivity is unavailable, the Go router bypasses the worker, Ollama, and NVIDIA NIM entirely, replays the pre-computed Golden Dataset payload, and the exact same four-route journey plays out identically, with zero live network dependency.

5. Implementation Plan — 48-Hour Sprint Execution

Runbook

5.1 Team Allocation

Team Konoha executed this build as a two-person unit:

- **Hrithik Sankar R** — core engineering execution: Go router, Python worker, AI pipeline clients, Neo4j integration, and frontend implementation.
- **Nithish Chandra S** (Team Leader) — product strategy, problem-statement alignment, and pitch narrative.

5.2 Pre-Sprint Checklist (T-minus 24 Hours)

- Install Go 1.22+, Python 3.11+, Bun 1.1+, Docker.
- Install Ollama; pull the local text model: `ollama pull llama3.2:3b`.
- Generate an NVIDIA NIM API key; verify access to meta/llama-3.2-11b-vision-instruct.
- Curate the Golden Dataset: a clean manual explicitly mentioning known tags, and a matching schematic with those tags visually present.

5.3 Phase 1 — Infrastructure & Scaffolding (Hour 0–4)

Boot Neo4j, scaffold the Go router (`/api/v1/health`, `/api/v1/ingest`), scaffold the Python worker, scaffold the Next.js/Bun frontend with a health-check ping.

5.4 Phase 2 — Ingestion & AI Pipeline (Hour 4–16)

PyMuPDF chunking, Pillow image compression, Go ↔ Ollama client with deterministic JSON-object prompting and the sanitizeLLMJSON stripper, Go ↔ NVIDIA NIM client for spatial extraction. Checkpoint: Golden Dataset round-trips clean JSON from both engines.

5.5 Phase 3 — Knowledge Graph Construction (Hour 16–26)

Neo4j constraints/indices, the Go Cypher driver and UNWIND/MERGE bulk-insert transaction, `GET /api/v1/graph` and `GET /api/v1/node/{tag_id}/context`. Checkpoint: correct Equipment ↔ Rule ↔ Coordinate topology after a test ingest.

5.6 Phase 4 — Frontend Visualization (Hour 26–38)

3D force-graph with physics warmup tuning, 2D bounding-box canvas, intelligence drawer, shared

selection state. Checkpoint: clicking a node in 3D highlights the matching 2D bounding box and opens the drawer with the correct manual text.

5.7 Phase 5 — Hardening & Mock Mode (Hour 38–44)

MOCK_MODE circuit breaker, rate-limit semaphores, edge-case sweeps (empty spatial arrays, malformed LLM JSON, WebGL context loss).

5.8 Phase 6 — Demo & Submission (Hour 44–48)

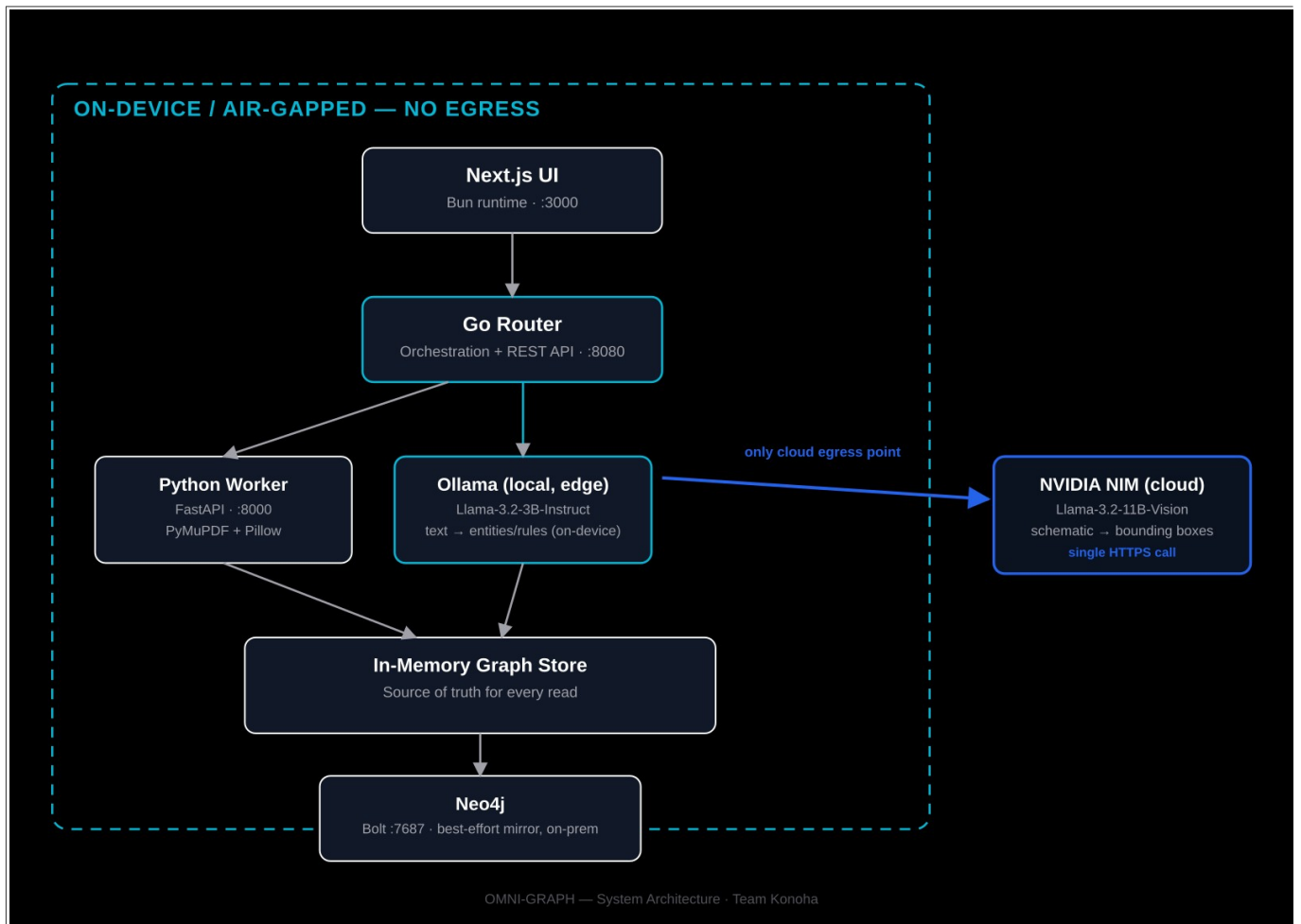
Pitch deck, one-take walkthrough recorded in Mock Mode, final README with architecture diagram, submission.

5.9 Beyond the Initial 48 Hours

With additional runway secured beyond the original sprint window, the team extended the build with production-grade hardening: real Neo4j dual-write validated end-to-end, live model calls exercised with real credentials (including a region-fallback fix after observing degenerate pixel-coordinate responses), backend/worker/frontend performance passes, and a full UI rework from a single three-pane dashboard into the four-route guided flow documented in Section 4.

6. Architecture Diagram

The full system architecture, repeated here for standalone reference. The dashed boundary is the entire Hybrid-Edge argument: everything inside it runs on-device with no outbound network calls; NVIDIA NIM is the one deliberate cloud egress point, reached over a single HTTPS call per ingest for spatial localization only.



END OF TECHNICAL SUBMISSION DOCUMENT